

Production-Grade Backend Technical Handbook

Go Todo API / Productivity + Finance Platform

Status: Living Engineering Handbook **Audience:** Backend Developers, Mobile Developers, Future Contributors, DevOps **Architecture Level:** Production-Grade Monolith (Go + Gin + PostgreSQL)

Purpose

This handbook serves as the single source of truth for the backend architecture, coding doctrine, extension rules, operational guidance, API integration strategy, and future development direction.

This document exists to prevent architectural drift, inconsistent implementations, duplicated logic, fragile integrations, and undocumented assumptions.

It defines how the system is built, how future code must be added, and how mobile/web clients are expected to interact with the backend.

Table of Contents

1. System Architecture Overview
 2. Sprint-by-Sprint Build History
 3. Engineering Doctrine
 4. Layer Responsibilities
 5. Authentication Architecture
 6. API Standards
 7. Database Architecture
 8. Middleware Architecture
 9. Scheduler Architecture
 10. Swagger / API Documentation Workflow
 11. Integration Testing Strategy
 12. Mobile Integration Handbook
 13. Feature Extension Blueprint
 14. Production Deployment Guide
 15. Operational Runbook
 16. Future Architecture Roadmap
-

1. System Architecture Overview

High-Level Architecture

```
Mobile App (React Native / Flutter)
    ↓
Web Frontend (future)
    ↓
REST API (Gin)
    ↓
Middleware Layer
    ↓
Handler Layer
    ↓
Service Layer
    ↓
Repository Layer
    ↓
PostgreSQL
```

Supporting Components:

- JWT Authentication
- Refresh Token Session Management
- Background Schedulers
- Swagger / OpenAPI Documentation
- Integration Testing Layer
- Structured Logging
- Security Middleware
- Health / Readiness Monitoring

Architectural Philosophy

Current architecture intentionally uses a **modular monolith**.

Rationale:

- simpler deployment
- lower operational complexity
- faster iteration speed
- easier debugging
- lower infrastructure cost
- sufficient scalability for early production stages

Microservices are intentionally deferred until real scaling pressure exists.

2. Sprint-by-Sprint Build History

Sprint 1 — Authentication Foundation

Implemented:

- user registration
- login
- JWT access tokens
- refresh token foundation
- logout
- auth middleware
- current user endpoint

Objective: Establish secure authentication baseline.

Sprint 2 — Productivity Core

Implemented:

- Todos CRUD
- Notes CRUD
- Note pinning

Objective: Build non-financial productivity core.

Sprint 3 — Dashboard

Implemented:

- dashboard summary
- aggregate counts
- quick overview endpoints

Objective: Provide user-facing summary insights.

Sprint 4 — Profile Management

Implemented:

- profile retrieval

- profile update
- password change

Objective: Account lifecycle management.

Sprint 5 — Hardening Foundation

Implemented:

- request validation improvements
- response standardization
- middleware consistency
- security cleanup

Objective: Strengthen platform consistency.

Sprint 6 — Finance Foundation

Implemented:

- transaction categories
- transactions CRUD
- finance summary

Objective: Establish financial domain foundation.

Sprint 7 — Finance Expansion

Implemented:

- budgets
- recurring transactions
- alerts
- analytics

Objective: Financial automation + planning.

Sprint 8 — Architecture Stabilization

Implemented:

- service cleanup

- repository consistency
- validation normalization
- transaction flow corrections

Objective: Reduce technical debt before production hardening.

Sprint 9 — Production Readiness

Implemented:

- session-based refresh token auth
- refresh rotation
- DB hardening
- Swagger/OpenAPI
- integration testing foundation
- structured middleware design
- graceful shutdown
- security headers
- request tracing

Objective: Production launch readiness.

3. Engineering Doctrine

Core rules are mandatory.

Violating these rules causes architecture drift.

Rule 1

Handlers must remain thin.

Rule 2

Business logic belongs in services.

Rule 3

Repositories only access persistence.

Rule 4

DTOs define request contracts.

Rule 5

Models represent persistence structures.

Rule 6

Responses must use shared response helpers.

Rule 7

No business logic inside middleware.

Rule 8

No direct DB queries from handlers.

Rule 9

No HTTP concerns inside repositories.

Rule 10

Auth logic must remain centralized.

4. Layer Responsibilities

Handler Layer

Allowed:

- request binding
- validation
- extracting headers
- reading auth context
- calling services
- formatting API responses

Forbidden:

- SQL queries
 - business rules
 - transaction orchestration
 - auth token generation
-

Service Layer

Allowed:

- domain rules
- orchestration
- auth workflows
- validation beyond DTO schema
- scheduler logic

Forbidden:

- gin.Context usage
 - HTTP response formatting
 - direct transport concerns
-

Repository Layer

Allowed:

- GORM queries
- persistence operations
- filtering
- joins
- pagination queries

Forbidden:

- business logic
 - JWT handling
 - HTTP semantics
-

5. Authentication Architecture

Access Token

Type: JWT

Purpose: short-lived API authorization

Characteristics:

- stateless
 - bearer token
 - included in Authorization header
 - validated by middleware
-

Refresh Token

Type: opaque random secret

NOT JWT.

Purpose: obtain new access tokens.

Characteristics:

- random secret
 - high entropy
 - hashed before DB storage
 - revocable
 - rotated after use
-

Session Storage

Table:

user_sessions

Contains:

- user_id
- refresh_token_hash

- user_agent
 - ip_address
 - expires_at
 - is_revoked
-

Refresh Rotation Flow

```
Client sends refresh token
↓
Server hashes token
↓
Lookup active session
↓
Revoke old session
↓
Generate new access token
↓
Generate new refresh token
↓
Create new session
↓
Return both tokens
```

Security benefit: replay attack resistance.

Logout Semantics

Logout revokes exact session.

Not all sessions.

Meaning:

```
logout on phone
phone session revoked
laptop remains logged in
```

6. API Standards

Success Response

```
{
  "success": true,
  "message": "operation successful",
  "data": {},
  "request_id": "req_xxx"
}
```

Error Response

```
{
  "success": false,
  "message": "validation failed",
  "errors": {},
  "request_id": "req_xxx"
}
```

Rules:

- never return arbitrary JSON shapes
- always use shared response helpers
- keep consistent envelope structure

7. Database Architecture

Core tables:

- users
- todos
- notes
- transaction_categories
- transactions
- budgets
- recurring_transactions
- alerts

- user_sessions

Integrity Rules

Mandatory:

- foreign keys
- unique constraints
- indexed filters
- soft deletes where appropriate

Examples:

```
budgets:  
UNIQUE(user_id, category_id, month)
```

```
transaction_categories:  
UNIQUE(user_id, name, type)
```

8. Middleware Architecture

Execution order:

```
Request ID  
↓  
Structured Logger  
↓  
Recovery  
↓  
CORS  
↓  
Security Headers  
↓  
Auth Middleware (protected routes)
```

Request ID

Purpose: traceability.

Structured Logger

Purpose: observability.

Recovery

Purpose: panic containment.

CORS

Purpose: frontend compatibility.

Security Headers

Purpose: basic hardening.

Auth Middleware

Purpose: JWT validation.

9. Scheduler Architecture

Current schedulers:

- recurring scheduler
- alert scheduler
- session cleanup scheduler

Pattern:

```
scheduler/feature_scheduler.go
```

Required structure:

- constructor
 - Start()
 - cron registration
 - bootstrap in main
-

10. Swagger Workflow

Documentation uses swaggo annotations.

Example:

- Summary
- Description
- Tags
- Params
- Success responses
- Failure responses
- Security requirements

Generation:

```
swag init
```

Docs endpoint:

```
/api/docs/index.html
```

11. Integration Testing Strategy

Approach: real integration testing.

NOT mock-heavy fake testing.

Flow:

```
HTTP request  
→ router  
→ middleware  
→ handler  
→ service  
→ repository  
→ postgres  
→ response
```

Current tested:

- register
- login

Future required:

- refresh
- logout
- /auth/me
- invalid token
- transaction CRUD
- budgets
- recurring
- analytics

12. Mobile Integration Handbook

Login

Mobile sends:

POST /auth/login

Receives:

- access token
- refresh token

Storage guidance:

Access token: short-term secure storage.

Refresh token: secure encrypted storage.

Avoid plaintext insecure storage.

Protected Requests

Use:

```
Authorization: Bearer access_token
```

Access Token Expiry

On 401:

```
call refresh endpoint  
replace tokens  
retry original request
```

Recommended: network interceptor.

Logout

Send refresh token.

Endpoint:

```
POST /auth/logout
```

Offline Strategy (Future)

Recommended:

- local cache
 - queued writes
 - sync reconciliation
-

13. Feature Extension Blueprint

When adding a feature:

Required structure:

- dto/
- model/

- repository/
- service/
- handler/
- swagger docs
- integration tests
- route registration

Checklist:

- validation rules
 - DB constraints
 - auth implications
 - mobile API contract
 - logging impact
 - tests
-

14. Production Deployment Guide

Required:

- release mode
 - env configuration
 - trusted proxies
 - SSL termination
 - health endpoint
 - readiness endpoint
 - scheduler monitoring
 - DB backups
 - secure secrets handling
-

15. Operational Runbook

When issues happen:

Auth failures:

- JWT secret mismatch
- expired refresh token
- revoked session

DB failures:

- connectivity

- migrations
- credential mismatch

Scheduler failures:

- cron startup missing
- panic recovery logs

Deployment failures:

- env missing
 - proxy misconfiguration
 - release mode mismatch
-

16. Future Roadmap

Phase 2:

- wallets
- transfers
- debt tracking
- loan engine
- subscriptions
- reports
- exports
- attachments

Phase 3:

- Redis caching
- background queue workers
- push notifications
- analytics warehouse
- admin portal
- RBAC

Phase 4:

Only if necessary:

- service decomposition
 - event-driven architecture
 - microservice extraction
-

Final Doctrine

Architecture must remain intentional.

New code must follow established patterns.

Consistency beats cleverness.

Scalability follows discipline.